

WINTERSEMESTER 2024/25

Studiengang/Abschluss: Wirtschaftsinformatik / B.Sc.

Prüfungsfach: Programmierung

Prüfungsnummer: **5706**

Prüfer: Prof. Dr. Andreas Biesdorf

Anzahl der Arbeitsblätter: 9 Seiten (ohne Deckblätter)

Bearbeitungszeit: 90 Minuten

Mit Bleistift oder in roter Farbe geschriebene Ausführungen werden nicht gewertet!

Zugelassene Hilfsmittel:

Das im Rahmen der Vorlesung erstellte Hilfsmittel.

Matrikelnummer:

--	--	--	--	--	--

Datum der Klausur:

.....

Platznummer:

Punkte: _____ von **90** möglichen

Note: _____

Handzeichen des Prüfers: _____

Hinweise zu Klausuren im FB Wirtschaft

- Mit Aufnahme der Prüfung bringen Sie zum Ausdruck, dass Sie sich für prüfungstauglich halten. Ein Rücktritt von der Klausur während der Bearbeitungszeit ist damit nur noch in ganz besonderen Ausnahmefällen möglich.
- Sollten Sie zu Beginn der Bearbeitungszeit prüfungstauglich sein, dann aber während der Bearbeitungszeit prüfungsuntauglich werden, müssen Sie die Klausur abbrechen, Ihre Prüfungsuntauglichkeit den Aufsichtführenden anzeigen und schnellstmöglich, in jedem Fall aber fristgerecht für ein ärztliches Attest sorgen.
- Mit der Abgabe Ihrer Klausur bringen Sie zum Ausdruck, dass Sie sich während der gesamten Bearbeitungszeit für prüfungstauglich gehalten haben. Ein Rücktritt ist dann nicht mehr möglich.
- Jacken und Taschen sind in ausreichender Entfernung vom Platz zu deponieren.
- Die Bearbeitungszeit beginnt erst, wenn alle Klausuren ausgeteilt sind.
- Es sind nur die zugelassenen Hilfsmittel zu verwenden.
- Das Verlassen des Raumes für Toilettengänge während der Klausur ist nur nach Meldung und Bestätigung der Aufsicht gestattet. Die vorübergehende Abwesenheit wird im Protokoll vermerkt.
- Es sind keine anderen als die ausgegebenen Bearbeitungsblätter zu benutzen.
- Auf dem Tisch sind nur Schreibutensilien erlaubt, keine Mäppchen.
- Jeder Teilnehmer muss sich mit einem Studierendenausweis oder Lichtbildausweis ausweisen.
- Geräusche und Störungen jeder Art sind zu vermeiden.
- Die Klausurensätze sind grundsätzlich nicht zu öffnen. Wer den Klausurensatz öffnet, trägt dafür Sorge, dass die Blätter (innerhalb der Bearbeitungszeit) sortiert und neu geheftet werden. Das Risiko des Verlustes von einzelnen Blättern trägt der Studierende!
- Es ist nicht erlaubt, Aufgabenstellungen auf gesondertes Papier abzuschreiben.
- Mobiltelefone, Smartphones und Smartwatches sind nicht erlaubt!
- Alle Studierenden bleiben auf ihren Plätzen und verhalten sich ruhig, bis die Klausuren vollständig eingesammelt sind. Folgen Sie den Anweisungen der Aufsichtführenden.
- Bei Täuschung oder versuchter Täuschung wird mit „nicht ausreichend“ bewertet.

Die Klausur besteht aus 6 Aufgaben zu den in der Lehrveranstaltung behandelten Themenbereichen. Die Punktzahl ist bei jeder Aufgabe mit angegeben.

Aufgabe	Mögliche Punkte	Erreichte Punkte
A1	15	
A2	25	
A3	10	
A4	10	
A5	20	
A6	10	
Summe	90	
Evtl. Bonus		

Praxisprojekt in PROG in Semester _____ eingereicht ☐ Nein ☐ Ja

Bitte dokumentieren Sie die Antworten zu allen Aufgaben in der entsprechenden Textbox auf dem Blatt oder der entsprechend benannten Datei (z.B. aufgabe1.txt oder aufgabe5.py) in dem Ordner auf der VM.

Aufgabe 1

- 1) In realen Software Engineering-Projekten gibt es i.d.R. fünf zentrale Stakeholder-Gruppen, die miteinander in Beziehung stehen. Bitte benennen Sie die fünf Stakeholder-Gruppen und erläutern Sie ihre Beziehung zueinander. Erstellen Sie gerne eine Skizze der Beziehungen. [7 Punkte]

- 2) Ein Unternehmen entwickelt eine umfangreiche Finanzanalyse-Software. Das Management fordert eine Markteinführung binnen 6 Monaten, stellt ein Budget von maximal 500.000 € zur Verfügung und erwartet gleichzeitig höchste Qualitätsstandards. Bewerten Sie diese Vorgaben unter Berücksichtigung des **Magischen Dreiecks** des Projektmanagements. Gehen Sie dabei auf die potenziellen Zielkonflikte und Auswirkungen auf das Projekt ein. (3 Punkte)

- 3) Ein Unternehmen entwickelt eine Cloud-basierte Plattform für algorithmischen Hochfrequenzhandel. Die Software muss Finanzmarktdaten in Echtzeit analysieren, automatisierte Handelsentscheidungen treffen und Transaktionen in Millisekunden ausführen. Leiten Sie jeweils drei aus Ihrer Sicht wichtigen funktionalen und nicht-funktionalen Anforderungen für dieses System ab und erläutern Sie diese stichwortartig. (5 Punkte)

Aufgabe 2

- 1) Was versteht man beim Testen unter „Path-complete“ und bei welcher allgemeinen Kategorie von Tests spielt dies eine Rolle? (4 Punkte)

- 2) Wie trägt **defensive Programmierung** zur Erhöhung der Softwarequalität bei? Was sind typische Maßnahmen, die im Rahmen der defensiven Programmierung eingesetzt werden? (5 Punkte)

3) Gegeben sei folgendes Programm:

prog_ws2425/src/aufgabe02.py

```
def minOfThree(a, b, c):  
    """  
    a, b und c sind Zahlen  
    Gibt das Minimum von a, b und c zurück  
    """  
    if a < b:  
        kleiner = a  
    else:  
        kleiner = b  
  
    if c < kleiner:  
        kleiner = c  
  
    return kleiner
```

Die Funktion ist mit einem White-Box-Test zu prüfen. Welche Pfade sollten hierfür betrachtet werden? Bitte benennen und erläutern Sie diese kurz. **[4 Punkte]**

Erstellen Sie in der Datei prog_ws2425/test/test_aufgabe02.py die korrespondierenden Tests mithilfe von pytest. **[2 Punkte]**

4) Gegeben sei folgendes Programm:

prog_ws2425/src/aufgabe02.py

```
def ggt(a, b):  
    """  
    Berechnet den größten gemeinsamen Teiler (GGT) zweier Zahlen  
    a und b müssen ganze Zahlen größer als 0 sein.  
    Gibt den GGT von a und b zurück.  
    """  
    if b == 0:  
        return a  
    else:  
        return ggt(b, a % b)  
  
# Beispielaufruf  
print(ggt(18, 27))  
print(ggt(42, 70))
```

Die Funktion ist mit einem White-Box-Test zu prüfen. Welche Pfade müssen für eine vollständige Prüfung betrachtet werden? Bitte benennen und erläutern Sie diese kurz.
[6 Punkte]

Ergänzen Sie in der Datei prog_ws24/test/test_aufgabe02.py die korrespondierenden Tests mithilfe von pytest. **[4 Punkte]**

Aufgabe 3

Betrachten Sie folgende Funktion `fehlerhafte_berechnung`, die eine Liste von Zahlen und einen Index als Parameter entgegennimmt. Die Funktion soll den Quotienten zwischen der Zahl in der Liste am gegebenen Index und einem vom Nutzer eingegebenen Wert zurückgeben. **[10 Punkte]**

prog_ws2425/src/aufgabe03.py

```
def fehlerhafte_berechnung(liste, index):  
    """  
    Nimmt an: liste ist eine Liste von Zahlen,  
              index ist die Position des zu teilenden Werts.  
    Gibt zurück: Den Wert an der gegebenen Indexposition,  
                 geteilt durch eine Benutzereingabe.  
    """  
    wert = liste[index]  
  
    eingabe = input("Geben Sie eine Zahl zum Dividieren ein: ")  
    zahl = int(eingabe)  
  
    ergebnis = wert / zahl  
  
    print("Ergebnis:", ergebnis)  
  
    return ergebnis  
  
# Beispielaufruf  
liste = [10, 20, 30, 40]  
index = int(input("Geben Sie den Index ein: "))  
fehlerhafte_berechnung(liste, index)
```

Die Funktion `fehlerhafte_berechnung` enthält mehrere potentielle Exceptions, die nicht behandelt werden. Überarbeiten Sie den Code:

- Falls der Benutzer einen ungültigen Index angibt, soll dies mithilfe einer Exception identifiziert werden und eine Fehlermeldung ausgegeben werden.
- Falls die Benutzereingabe keine gültige Zahl ist, soll dies mithilfe einer Exception identifiziert werden und eine entsprechende Fehlermeldung ausgegeben werden.
- Falls der Benutzer eine 0 eingibt, soll dies mithilfe der entsprechenden Exception identifiziert werden und das Programm eine geeignete Fehlermeldung ausgeben.
- Falls keine Exception auftritt, soll das Ergebnis normal berechnet und ausgegeben werden.
- Am Ende der Funktion soll immer eine Nachricht ausgegeben werden, die bestätigt, dass die Berechnung abgeschlossen wurde – unabhängig davon, ob eine Exception aufgetreten ist oder nicht.

Aufgabe 4

Erstellen Sie unter Verwendung des getopt-Frameworks ein Programm, das über die Kommandozeile mit bestimmten Parametern gestartet wird. Das Programm soll eine einfache Berechnung durchführen und das Ergebnis in eine Datei speichern.

[10 Punkte]

Anforderungen:

- Das Programm akzeptiert zwei Zahlen [-a und -b] sowie eine Rechenoperation [-o] als Kommandozeilenargumente. Weiterhin soll mit -f der Dateiname für das Ergebnis angegeben werden.
- Unterstützte Operationen sind:
 - add (Addition)
 - sub (Subtraktion)
 - mul (Multiplikation)
 - div (Division, mit Fehlerbehandlung für 0)
- Das Ergebnis der Berechnung soll in eine Datei [ergebnis.txt] geschrieben werden.
- Falls die Argumente fehlen oder fehlerhaft sind, soll eine passende Fehlermeldung ausgegeben werden.

Beispielaufruf:

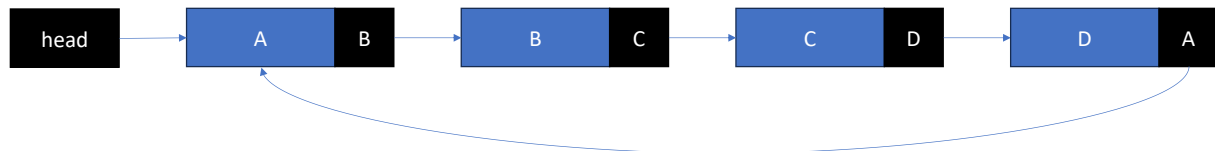
```
python3 aufgabe04.py -a 10 -b 5 -o add -f ergebnis.txt
```

Erwarteter Text in der Datei ergebnis.txt:

```
Ergebnis: 15
```

Aufgabe 5

Eine kreisverkettete Liste ist eine Variante der einfach verketteten Liste, bei der das letzte Element auf das erste Element zurückverweist, wodurch eine Schleifenstruktur entsteht. Die folgende Abbildung zeigt ein Beispiel einer kreisverketteten Liste mit vier Elementen:



Gegeben ist der folgende unvollständige Code der Klasse `CircularLinkedList`:

prog_ws2425/src/aufgabe05.py

```

class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class CircularLinkedList:
    def __init__(self):
        self.head = None

    def insert(self, key):
        pass

    def delete(self, key):
        pass

    def display(self):
        pass
  
```

- Ergänzen Sie eine Methode `insert(self, key)`, die ein neues Element am Ende der Liste einfügt und den Zeiger des letzten Elements auf das erste Element setzt. **[10 Punkte]**
- Ergänzen Sie eine Methode `delete(self, key)`, die ein Element mit dem Wert `key` aus der Liste entfernt. **[5 Punkte]**
- Implementieren Sie eine Methode `display(self)`, die die Liste in der richtigen Reihenfolge ausgibt, ohne dass ein unendlicher Loop entsteht. **[5 Punkte]**

Aufgabe 6

Gegeben sei eine Liste von **Dokumenten**, die jeweils einen **Titel** und einen **Inhalt** enthalten. Implementieren Sie eine Funktion `quicksort_documents(documents)`, die die Liste der Dokumente **zuerst nach der Länge des Inhalts (gemessen in Zeichen) und dann alphabetisch nach dem Titel** sortiert. **[10 Punkte]**

- Falls zwei Dokumente gleich lang sind, soll die alphabetische Reihenfolge des Titels entscheiden.
Hinweis: Texte können in Python mit `<`, `>`, `=` alphabetisch verglichen werden
- Die Funktion soll das **Quicksort-Verfahren** verwenden und die Dokumente entsprechend den Vergleichskriterien vergleichen.
- Die Dokumente sind als **Liste von Tupeln** gegeben, wobei jedes Tupel den **Titel** und den **Inhalt** des Dokuments enthält.

`prog_ws2425/src/aufgabe06.py`

```
def quicksort_documents(documents):  
    pass  
  
documents = [  
    ("Document A", "Dies ist der Inhalt von Dokument A."),  
    ("Document B", "Dies ist der Inhalt von Dokument B."),  
    ("Document C", "Dies ist der längere Inhalt von Dokument C."),  
    ("Document D", "Dies ist der längere Inhalt von Dokument D."),  
    ("Document E", "Dies ist der Inhalt von Dokument E."),  
    ("Document F", "Inhalt von F.")  
]  
  
sorted_documents = quicksort_documents(documents)  
  
for document in sorted_documents:  
    print(document)
```

Erwartete Ausgabe:

```
('Document F', 'Inhalt von F.')  
( 'Document A', 'Dies ist der Inhalt von Dokument A.')  
( 'Document B', 'Dies ist der Inhalt von Dokument B.')  
( 'Document E', 'Dies ist der Inhalt von Dokument E.')  
( 'Document C', 'Dies ist der längere Inhalt von Dokument C.')  
( 'Document D', 'Dies ist der längere Inhalt von Dokument D.')
```